

CS 6008 Network Security - Secure Chat Application Report

Name : Rajas Paunikar (25M2126) Date : 4th September 2026

Overview

This report documents the design, implementation, and security evaluation of a one-to-one secure chat application built over TCP sockets in C++. The application was developed incrementally across five phases, with each phase introducing exactly one new security property on top of the previous phase's protocol.

Phase 1

Establishes a working client-server chat protocol with no security at all, serving as a deliberately insecure baseline.

Phase 2

Introduces confidentiality between each client and the server using a from scratch Diffie-Hellman key exchange (RFC 3526 Group 14) and AES-256-GCM authenticated encryption.

Phase 3

Adds server authentication through a minimal PKI , a self-signed Certificate Authority, server certificate issuance, client-side certificate validation, and a challenge response proof of possession mechanism closing the man-in-the-middle gap left open by Phase 2's unauthenticated key exchange.

Phase 4

Layers end-to-end encryption directly between the two communicating clients, independent of and undecryptable by the server, so that the relay itself cannot read message content even though it routes every message.

Phase 5

Extends this end-to-end session with automatic key rotation every 60 seconds, providing forward secrecy so that a future compromise of any single key exposes only the messages sent during that key's brief lifetime.

VM Topology

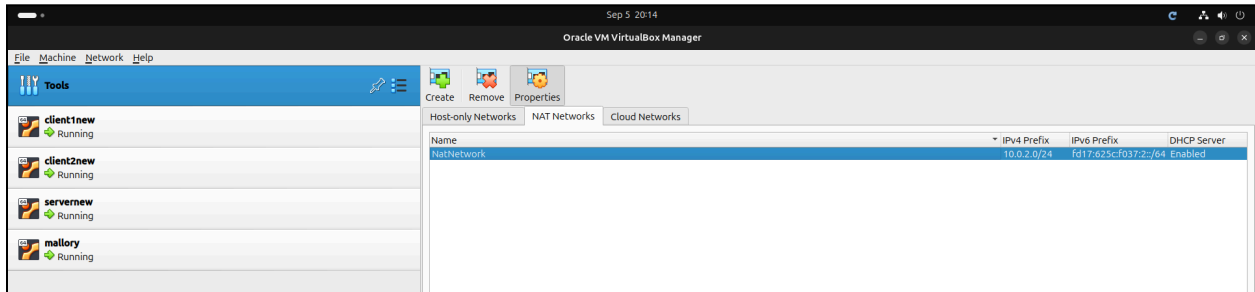


Fig1. Showing 4 VM's present each in a NAT Network configuration

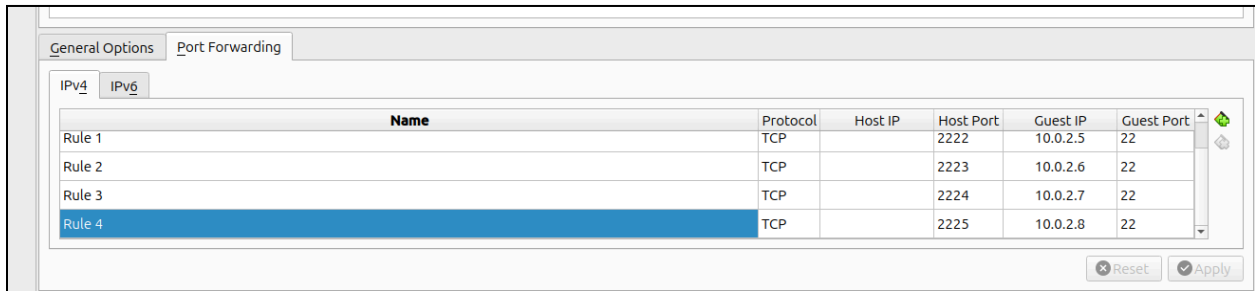


Fig2. Showing 4 VM's IP with port forwarding setup for easy ssh from host machine

VM	Role	IP Address
Server VM	server	10.0.2.6
Client1 VM	client	10.0.2.5
Client2 VM	client	10.0.2.7
Mallory	adversary	10.0.2.8

Phase 1 - Baseline Chat Application

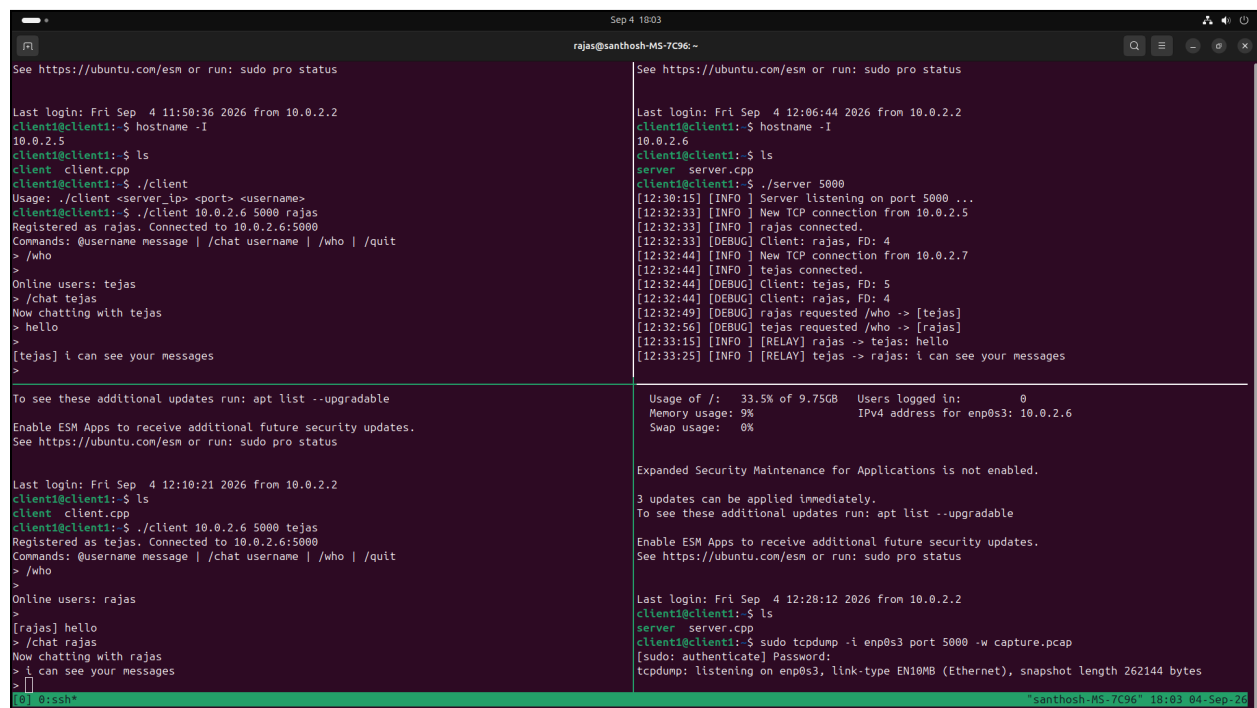
Protocol Description

Message framing. Every message exchanged between a client and the server is a length prefixed frame: a 4-byte header specifying the exact length of the payload, in network byte order, followed by the payload itself.

The **payload** itself is a pipe delimited text command of the form **TYPE|field1|field2**, for example **REGISTER|alice** , **MSG|bob|hello there** , **WHO|** , **WHO_RESP|bob ,alice** , and **QUIT|**.

Determining when a message is complete. Our protocol resolves this ambiguity using the length prefix header described above: the receiving side first reads exactly 4 bytes (looping over **recv()** as many times as necessary, since a single call is not guaranteed to return everything requested), decodes those bytes as the payload length, and then reads exactly that many further bytes to obtain the complete payload. Because the receiver never requests more bytes than it currently needs, it can never accidentally consume bytes belonging to the next frame, regardless of how the operating system happens to batch or split the underlying TCP segments.

Verification



```
Sep 4 18:03
rajas@santhosh-MS-7C96: ~
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Sep  4 11:50:36 2026 from 10.0.2.2
client@clienti:~$ hostname -I
10.0.2.5
client@clienti:~$ ls
client  client.cpp
client@clienti:~$ ./client
Usage: ./client <server_ip> <port> <username>
client@clienti:~$ ./client 10.0.2.6 5000 rajas
Registered as rajas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
>
Online users: tejas
> /chat tejas
Now chatting with tejas
> hello
>
[tejas] I can see your messages
>

To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Sep  4 12:10:21 2026 from 10.0.2.2
client@clienti:~$ ls
client  client.cpp
client@clienti:~$ ./client 10.0.2.6 5000 tejas
Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
>
Online users: rajas
>
[rajas] hello
> /chat rajas
Now chatting with rajas
> I can see your messages
>
[0] 0:ssh*
```

```
Sep 4 18:03
rajas@santhosh-MS-7C96: ~
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Sep  4 12:06:44 2026 from 10.0.2.2
client@clienti:~$ hostname -I
10.0.2.6
client@clienti:~$ ls
server  server.cpp
client@clienti:~$ ./server 5000
[12:30:15] [INFO] Server listening on port 5000 ...
[12:32:33] [INFO] New TCP connection from 10.0.2.5
[12:32:33] [INFO] rajas connected.
[12:32:33] [DEBUG] Client: rajas, FD: 4
[12:32:44] [INFO] New TCP connection from 10.0.2.7
[12:32:44] [INFO] tejas connected.
[12:32:44] [DEBUG] Client: tejas, FD: 5
[12:32:44] [DEBUG] Client: rajas, FD: 4
[12:32:49] [DEBUG] rajas requested /who -> [tejas]
[12:32:56] [DEBUG] tejas requested /who -> [rajas]
[12:33:15] [INFO] [RELAY] rajas -> tejas: hello
[12:33:25] [INFO] [RELAY] tejas -> rajas: I can see your messages

Usage of /: 33.5% of 9.75GB  Users logged in: 0
Memory usage: 9%          IPv4 address for enp0s3: 10.0.2.6
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Sep  4 12:28:12 2026 from 10.0.2.2
client@clienti:~$ ls
server  server.cpp
client@clienti:~$ sudo tcpdump -i enp0s3 port 5000 -w capture.pcap
[sudo: authenticate] Password:
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Fig3. Server log showing plain text content of relayed messages

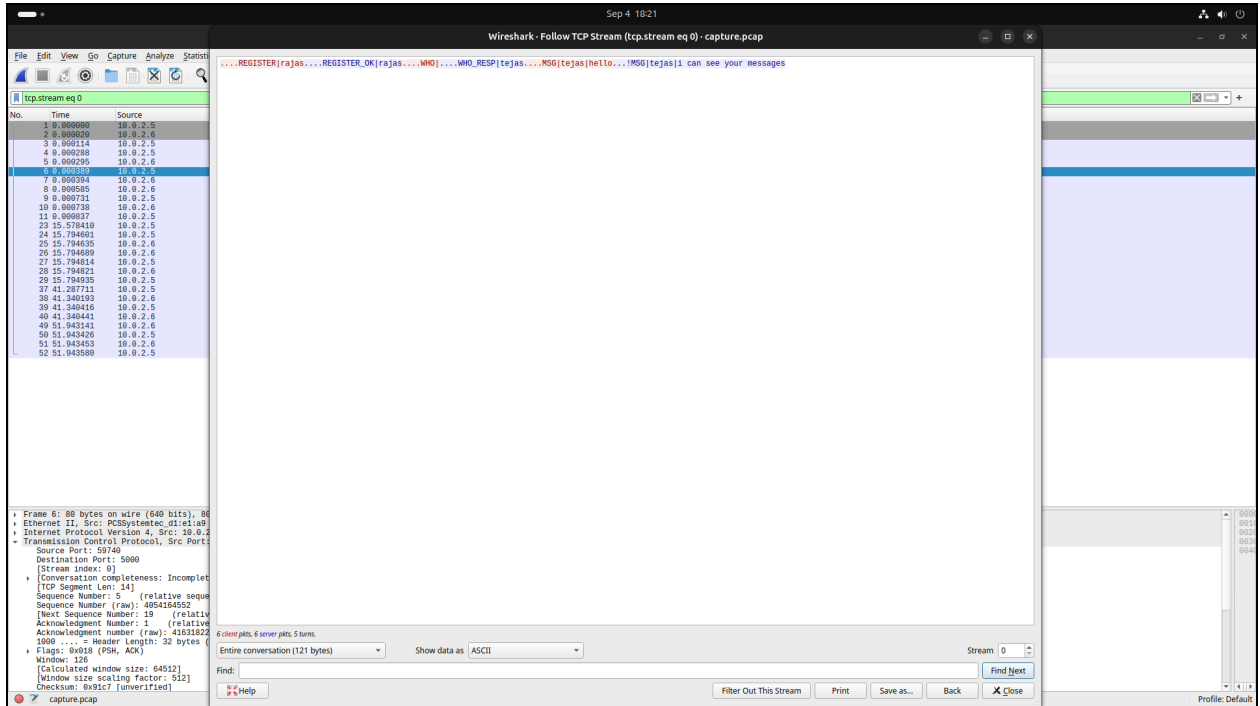


Fig4. Wireshark “Follow TCP Stream” showing message content visible in plain text

Source Code

phase_1

- client
- client.cpp
- README.md
- screenshots
 - Follow TCP Stream SS.png
 - wireshark_showing_all_communication.png
 - wireshark_showing_message.png
 - working_1.png
 - working_2.png
- server
- server.cpp

2 directories, 10 files

Phase 2 - Client Server confidentiality via Diffie Hellman

DH Implementation

Group selection

1. The exchange uses **RFC 3526 Group 14, a standard**, publicly published 2048-bit MODP (Modular Exponential) group, with generator $g=2$. The prime and generator are hard-coded as constants (`dh_group_14.h`) rather than computed at runtime.
2. A standard group was used instead of a self generated one because constructing a cryptographically sound prime for Diffie Hellman is genuinely difficult to get right. An attacker's search space for recovering a private exponent collapses dramatically if prime and generator are not perfect. For example, in a naively chosen group with $p=7$, $g=2$ the generator only cycles through a 3-element subgroup $\{2,4,1\}$ instead of the full range of six values, making brute-force recovery of a private exponent trivial.

Key Generation and Exchange

Each party generates a private exponent uniformly at random in the range $[0,p)$ using `BN_rand_range`, then computes the corresponding public value via `BN_mod_exp`. Only the public value is ever transmitted, the private exponent never leaves the machine that generated it. Upon receiving the peer's public value, each side computes the shared secret as Key which is the same.

Independent exchange per client

Each connecting client performs its own complete DH exchange with the server, using freshly generated random exponents, the server does not reuse a single exchange across multiple clients.

Questions on DH implementation

Why should we hash the [raw DH shared secret before using it as a symmetric] key?

1. Wrong size and shape AES-256 requires exactly 256 bit key, but raw DH is a number bounded by size p .
2. Raw secret is not uniformly random, using a structured data directly as a key risks exposing subtle biases or correlations.

3. Passing raw secret through cryptographic hash function solves both the above problems , it resizes and produces output that is indistinguishable. SHA-256 is one way hence it can't be reversed to obtain the raw key.

Briefly explain how you derived the AES key and why AES-GCM was used rather than plain AES.

1. The AES-256 key for each connection is derived by hashing the raw DH shared secret's bytes with SHA-256.
2. In a case where an adversary may change the encrypted message , there is no mechanism to determine if the message integrity has been compromised or not.
3. During encryption with AES-GCM 16 byte tag is computed and sent.
4. On decryption, this tag is recomputed and compared against the tag that arrived with the ciphertext; any modification to even a single bit of the ciphertext changes the recomputed tag, causing decryption to fail outright rather than silently returning corrupted plaintext.

Verification

```

Registered as rajas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
>
[Disconnected from server]
^C
client@client1:~/phase_2$ ./client 10.0.2.6 5000 rajas
DH shared secret fingerprint: 4e2ffbd155152e38
Registered as rajas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
>
Online users: tejas
> /chat tejas
Now chatting with tejas
> hello my name is rajas
[tejas] hello my name is tejas , how are you ?
>
[tejas] checking
> checking
> /quit
client@client1:~/phase_2$

client@client1:~$ cd phase_2/
client@client1:~/phase_2$ ./client 10.0.2.6 5000 tejas
DH shared secret fingerprint: bcc0abf132286c9d
Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
> /who
Online users: rajas
>
Online users: rajas
>
[rajas] hello my name is rajas
> /chat rajas
Now chatting with rajas
> hello my name is tejas , how are you ?
> checking
[rajas] checking
> /quit
client@client1:~/phase_2$

rajas@10.129.131.167's password:
capture.pcap 100% 4562 7.1MB/s 00:00
client@client1:~$ sudo tcpdump -i enp0s3 port 5000 -w capture_phase2.pcap
[sudo: authenticate] Password:
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C37 packets captured
40 packets received by filter
0 packets dropped by kernel
^Cclient@client1:~$
client@client1:~$ ls
capture.pcap capture_phase2.pcap phase_2 server server.cpp
client@client1:~$ rm -rf capture_phase2.pcap
client@client1:~$ sudo tcpdump -i enp0s3 port 5000 -w capture_phase2.pcap
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C93 packets captured
95 packets received by filter
0 packets dropped by kernel
client@client1:~$ scp capture_phase2.pcap rajas@10.129.131.167:
rajas@10.129.131.167's password:
capture_phase2.pcap 100% 11KB 14.3MB/s 00:00
client@client1:~$

```

Fig5. Fingerprint printed on both client and server showing they match


```

phase 2 > test_crypto.tampering.cpp > (main)
19  std::string unencrypted;
20  if (!aes_gcm_decrypt(key, blob, decrypted)) {
21      std::cout << "Decryption failed unexpectedly\n";
22      return 1;
23  }
24  std::cout << "Decrypted plaintext: " << decrypted << "\n";
25  std::cout << (decrypted == plaintext ? "ROUND TRIP OK\n\n" : "ROUND TRIP MISMATCH\n\n");
26
27  std::string tampered = blob;
28  tampered[tampered.size() - 1] ^= 0xFF;
29
30  std::string tampered_result;
31  bool tamper_decrypt_ok = aes_gcm_decrypt(key, tampered, tampered_result);
32
33  std::cout << "Tamper test: Flipped last byte of ciphertext.\n";
34  if (!tamper_decrypt_ok) {
35      std::cout << "TAMPER CORRECTLY REJECTED (GCM authentication failed as expected)\n";
36  } else {
37      std::cout << "TAMPER NOT DETECTED - THIS IS A BUG!\n";
38  }
39
rajas@santhosh-MS-7C96:~/Desktop/sem3/Network&Security/Assignment1/phase_2$ g++ test_crypto.cpp -o test_crypto
/usr/bin/ld: test_crypto.cpp:(.text+0x10b): undefined reference to `aes_gcm_decrypt(unsigned char const*, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const
collect2: error: ld returned 1 exit status
rajas@santhosh-MS-7C96:~/Desktop/sem3/Network&Security/Assignment1/phase_2$ g++ test_crypto.cpp -o test_crypto -lcrypto
/usr/bin/ld: /tmp/cc8KX8e1.o: in function `main':
test_crypto.cpp:(.text+0xb7): undefined reference to `aes_gcm_encrypt(unsigned char const*, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
1:basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
6, std::__cxx11::basic_string_char, std::char_traits<char>, std::allocator<char>) > const&
collect2: error: ld returned 1 exit status
rajas@santhosh-MS-7C96:~/Desktop/sem3/Network&Security/Assignment1/phase_2$ g++ crypto.cpp test_crypto.cpp -o test_crypto -lcrypto
Original plaintext: MSG|bob|hello this is a test message
Encrypted blob size: 64 bytes
Decrypted plaintext: MSG|bob|hello this is a test message
ROUND TRIP OK

Tamper test: flipped last byte of ciphertext.
TAMPER CORRECTLY REJECTED (GCM authentication failed as expected)
rajas@santhosh-MS-7C96:~/Desktop/sem3/Network&Security/Assignment1/phase_2$

```

Fig8. Tamper Detection test output, single byte flipped cipher text is rejected

Man In the Middle Attack

To demonstrate that Diffie-Hellman alone provides no authentication, it establishes a shared secret with whoever answers the connection, regardless of their identity a standalone proxy program ([mitm_proxy.cpp](#)) was built to actively intercept and read supposedly encrypted traffic between a victim client and the real server.

The proxy performs two entirely independent DH exchanges

1. It listens for an incoming connection from the victim client, and completes a DH handshake **posing as the server** generating its own keypair and deriving a shared secret with the victim, exactly as the real server would.
2. It separately opens an outbound connection to the real server, and completes a second DH handshake **posing as an ordinary client** deriving a second, entirely unrelated shared secret with the real server.

Setup

The victim client was configured to connect directly to the proxy's address and port instead of the real server's, which the assignment specification explicitly permits as sufficient to demonstrate the core vulnerability

```

Usage: ./client <server_ip> <port> <username>
client1@client1:~/phase_2$ ./client 10.0.2.8 6000
raj
DH shared secret fingerprint: 8343563c16f9e062
Registered as rajas. Connected to 10.0.2.8:6000
Commands: @username message | /chat username | /who | /quit
Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
Online users: (none)
/who
Online users: tejas
/chat tejas
Now chatting with tejas
[rajas] hello
/chat rajas
Now chatting with rajas
hello rajas
/quit
client1@client1:~/phase_2$

[16:14:44] [INFO] DH shared secret fingerprint: 75f522003cbeb21e
[16:14:50] [INFO] rajas connected.
[16:14:50] [DEBUG] Client: rajas, FD: 4
[16:15:03] [DEBUG] rajas requested /who -> []
[16:15:39] [INFO] New TCP connection from 10.0.2.5
[16:15:39] [INFO] DH shared secret fingerprint: 4b840bdd522032b1
[16:15:39] [INFO] tejas connected.
[16:15:39] [DEBUG] Client: tejas, FD: 5
[16:15:46] [DEBUG] Client: rajas, FD: 4
[16:15:46] [DEBUG] tejas requested /who -> [rajas]
[16:16:07] [DEBUG] rajas requested /who -> [tejas]
[16:16:17] [INFO] [RELAY] rajas -> tejas: hello
[16:16:34] [INFO] [RELAY] tejas -> rajas: hello rajas
[16:16:40] [INFO] tejas sent QUIT.
[16:16:40] [INFO] tejas connection closed.
[16:16:40] [INFO] tejas disconnected.
[16:16:40] [DEBUG] Client: rajas, FD: 4
[16:16:43] [INFO] rajas sent QUIT.
[16:16:43] [INFO] rajas connection closed.
[16:16:43] [INFO] rajas disconnected.
client1@client1:~/phase_2$

[MALLORY] Listening for victim on port 6000 ...
[MALLORY] Victim connected.
[MALLORY] DH #1 complete (posing as server, talking to victim). Fingerprint: 8343563c16f9e062
[MALLORY] Connected outbound to real server at 10.0.2.6:5000
[MALLORY] DH #2 complete (posing as client, talking to real server). Fingerprint: 75f522003cbeb21e
[MALLORY] Both handshakes complete. Two independent shared secrets established.
[MALLORY] Neither the victim nor the real server can tell they are talking to Mallory.
[MALLORY] Relaying traffic in both directions, decrypting and re-encrypting at each hop:

[MALLORY INTERCEPTED victim->server] REGISTER|rajas
[MALLORY INTERCEPTED server->victim] REGISTER_OK|rajas
[MALLORY INTERCEPTED victim->server] WHO|
[MALLORY INTERCEPTED server->victim] WHO_RESP|
[MALLORY INTERCEPTED victim->server] WHO|
[MALLORY INTERCEPTED server->victim] WHO_RESP|tejas
[MALLORY INTERCEPTED victim->server] MSG|tejas|hello
[MALLORY INTERCEPTED server->victim] MSG|tejas|hello rajas
[MALLORY INTERCEPTED victim->server] QUIT|
[MALLORY] Session ended.
client1@client1:~/phase_2$

client1@client1:~$ sudo tcpdump -i enp0s3 port 5000 -w capture_phase2_mitm.pcap
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
client1@client1:~$ ls
capture.pcap capture_phase2.pcap capture_phase2_mitm.pcap phase_2_server server.cpp
client1@client1:~$ rm -rf capture_phase2_mitm.pcap
client1@client1:~$ sudo tcpdump -i enp0s3 port 6000 -w capture_phase2_mitm.pcap
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
client1@client1:~$ ls
capture.pcap capture_phase2.pcap capture_phase2_mitm.pcap phase_2_server server.cpp
client1@client1:~$ scp capture_phase2_mitm.pcap rajas@10.129.131.167:
rajas@10.129.131.167's password:
capture_phase2_mitm.pcap 100% 24 45.9KB/s 00:00
client1@client1:~$

```

Fig 9. Mallory's log showing two independent DH fingerprints and decrypted plain text , victim client showing a normal looking session with no errors.

Result

The attack succeeded completely: the victim's client registered, exchanged chat messages, and disconnected with no errors or any indication that anything was amiss, while the proxy's log recorded the full plaintext of every message , including the registration exchange and real chat content , despite the victim's AES-GCM encryption functioning correctly on its own end throughout.

Question on MITM

Explain what observable evidence (if any) would have allowed the victim to detect that this attack was happening, and why an ordinary user would be unlikely to notice it.

The additional processing overhead of the proxy decrypting, logging, and re-encrypting every message introduces a small amount of extra latency, which a sufficiently instrumented observer might detect as an unusual delay.

In practice, however, an ordinary user would be very unlikely to notice either of these signals, for several concrete reasons specific to this implementation:

1. The client prints the fingerprint but never verifies anything , just prints it.
2. Every subsequent step behaves identical to a legitimate session.
3. The latency overhead is negligible for a local or LAN based proxy.

Source Code

```
.
├── client
├── client.cpp
├── crypto.cpp
├── crypto.h
├── dh.cpp
├── dh_group_14.h
├── dh.h
├── mitm_proxy
├── mitm_proxy.cpp
├── README.md
├── screenshots
│   ├── Follow_TCP_stream_client_1.png
│   ├── Follow_TCP_stream_client_2.png
│   ├── mitm_working.png
│   ├── showing_same_shared_key_hashed.png
│   └── tampered_bit_not_accepted.png
├── server
├── server.cpp
├── test
├── test_crypto
├── test_crypto_tampering.cpp
└── test_dh.cpp
```

2 directories, 21 files

Server Authentication via PKI

CA and Certificate Setup

A minimal Public Key Infrastructure was established entirely via OpenSSL command-line tools ([setup_ca.sh](#)), consisting of the following steps:

Certificate Authority Creation

Shell

```
openssl genrsa -out ca_key.pem 4096
openssl req -x509 -new -nodes -key ca_key.pem -sha256 -days 365 \
  -subj "/C=IN/O=CS6008-Assignment1/CN=MyChatCA" \
  -out ca_cert.pem
```

1. The first command generates a 4096-bit RSA private key for the CA.
2. The second produces a **self-signed** root certificate directly (the `-x509` flag tells OpenSSL to output a certificate rather than a signing request), with the **Issuer** and **Subject** fields both set to `MyChatCA`, the defining property of a self-signed certificate, since there is no higher authority above the root to vouch for it.
3. This root certificate (`ca_cert.pem`) is distributed to every client ahead of time as their trusted anchor, out-of-band from the chat protocol itself.

Server Key Pair and Certificate Signing Request

Shell

```
openssl genrsa -out server_key.pem 2048
openssl req -new -key server_key.pem \
  -subj "/C=IN/O=CS6008-Assignment1/CN=ChatServer" \
  -out server.csr
```

1. The server generates its own, entirely separate 2048-bit RSA key pair.
2. A CSR packages the server's public key together with its claimed identity (`CN=ChatServer`) into a signing request, at this stage it is not yet a valid certificate, only a request asking the CA to vouch for this specific public key and identity pairing

CA Signs the CSR , producing the server certificate

Shell

```
openssl x509 -req -in server.csr -CA ca_cert.pem -CAkey ca_key.pem \  
-CAcreateserial -days 365 -sha256 \  
-out server_cert.pem
```

The CA uses its own private key to sign the contents of the CSR, producing `server_cert.pem` a certificate whose **Issuer** field is `MyChatCA` and whose **Subject** field is `ChatServer`.

Verification

Shell

```
openssl verify -CAfile ca_cert.pem server_cert.pem
```

This confirms the issued certificate's signature genuinely traces back to the CA before it is ever used in the application.

Certificate Exchange and Validation

First Exchange

Before any Diffie-Hellman activity takes place, the server sends its certificate to the connecting client as the very first message on the connection. The certificate (`server_cert.pem`) is loaded once at server startup and transmitted in PEM text form as the content of a length-prefixed frame (`CERT | <pem>`), using the same framing layer as every other message in the protocol.

Validation

Upon receiving this certificate, the client performs three checks before proceeding any further, all implemented in `cert.cpp`

Failure Behaviour

If any of these checks fails for any reason the client aborts the connection immediately, without sending a username, without proceeding to any DH exchange, and without transmitting any further information.

Proof Of Possession

Why is it needed ?

Certificate validation alone confirms that a certificate is legitimately signed by the trusted CA and identifies the expected server but a certificate is, by design, public data, meant to be freely shared with every connecting client.

This means certificate validation alone cannot distinguish the genuine server from an attacker who has merely obtained a *copy* of the real certificate file, without ever having access to the corresponding private key.

Proof of possession closes this specific gap by requiring the server to demonstrate, in real time, that it actually holds the private key matching the certificate's embedded public key, not merely a copy of the certificate itself.

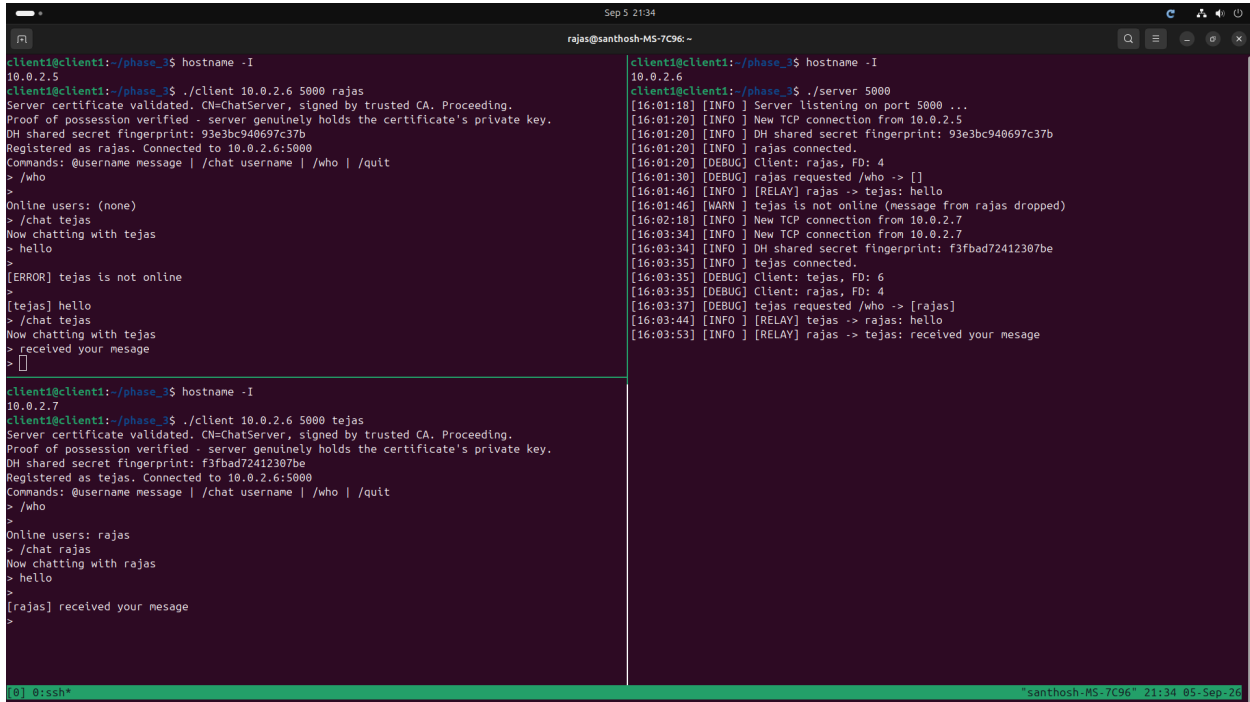
Mechanism

Immediately after the client successfully validates the server's certificate, it generates a fresh, random 16 byte nonce (via `RAND_bytes()`) and sends it to the server as a challenge (`CHALLENGE|<hex nonce>`).

The server signs this exact nonce using its private key.

The client then verifies this signature using the public key extracted directly from the certificate it already validated. If the signature does not verify, the client aborts immediately, before any DH exchange begins.

Verification - Legitimate Flow



```
client@client1:~/phase_1$ hostname -I
10.0.2.5
client@client1:~/phase_1$ ./client 10.0.2.6 5000 rajas
Server certificate validated. CN=ChatServer, signed by trusted CA. Proceeding.
Proof of possession verified - server genuinely holds the certificate's private key.
DH shared secret fingerprint: 93e3bc940697c37b
Registered as rajas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
Online users: (none)
> /chat tejas
Now chatting with tejas
> hello
[ERROR] tejas is not online
>
[tejas] hello
> /chat tejas
Now chatting with tejas
> received your message
>

client@client1:~/phase_1$ hostname -I
10.0.2.7
client@client1:~/phase_1$ ./client 10.0.2.6 5000 tejas
Server certificate validated. CN=ChatServer, signed by trusted CA. Proceeding.
Proof of possession verified - server genuinely holds the certificate's private key.
DH shared secret fingerprint: f3fbad72412307be
Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
Online users: rajas
> /chat rajas
Now chatting with rajas
> hello
[rajas] received your message
>

[16:01:18] [INFO] Server listening on port 5000 ...
[16:01:20] [INFO] New TCP connection from 10.0.2.5
[16:01:20] [INFO] DH shared secret fingerprint: 93e3bc940697c37b
[16:01:20] [INFO] rajas connected.
[16:01:20] [DEBUG] Client: rajas, FD: 4
[16:01:30] [DEBUG] rajas requested /who -> []
[16:01:46] [INFO] [RELAY] rajas -> tejas: hello
[16:01:46] [WARN] tejas is not online (message from rajas dropped)
[16:02:18] [INFO] New TCP connection from 10.0.2.7
[16:03:34] [INFO] New TCP connection from 10.0.2.7
[16:03:34] [INFO] DH shared secret fingerprint: f3fbad72412307be
[16:03:35] [INFO] tejas connected.
[16:03:35] [DEBUG] Client: tejas, FD: 6
[16:03:35] [DEBUG] Client: rajas, FD: 4
[16:03:37] [DEBUG] tejas requested /who -> [rajas]
[16:03:44] [INFO] [RELAY] tejas -> rajas: hello
[16:03:53] [INFO] [RELAY] rajas -> tejas: received your message
```

Fig 10. Full Legitimate Flow - Certificate verified , proof of possession verified , DH proceeds , chat working

MITM - Reattempt

Setup

The re-attempt used the manual-redirect configuration explicitly permitted by the assignment specification , the victim client was configured to connect directly to Mallory's address and port instead of the real server's.

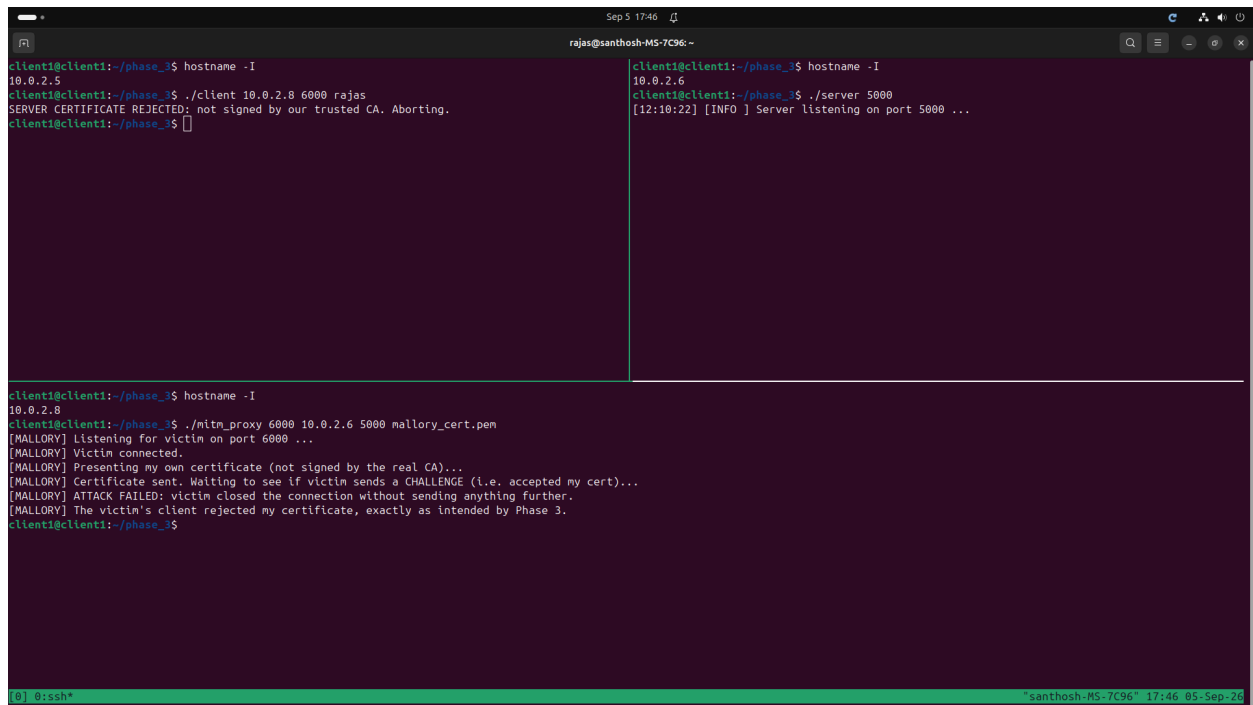
Attack Attempt

The Phase 2 MITM proxy ([mitm_proxy.cpp](#)) was extended to additionally present a certificate to the victim before attempting its DH handshake, since the victim's client now refuses to proceed at all without one.

Since Mallory has no access to the CA's private key, she cannot produce a certificate that will validate; this was tested with a self-signed certificate generated independently, sharing the identical Common Name ([ChatServer](#)) as the legitimate server but signed by no trusted authority.

Result

The attack now fails. The victim's client rejected the certificate immediately, printing **SERVER CERTIFICATE REJECTED: not signed by our trusted CA. Aborting.**



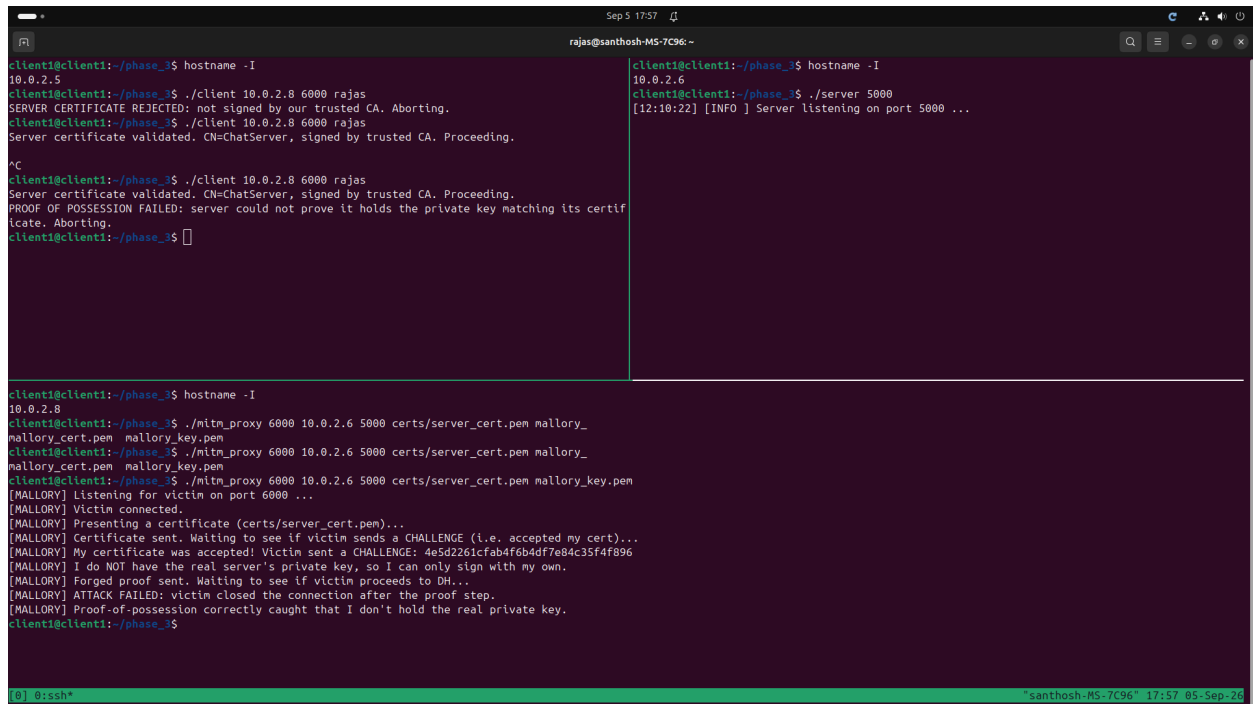
```
client1@client1:~/phase_3$ hostname -I
10.0.2.5
client1@client1:~/phase_3$ ./client 10.0.2.8 6000 rajas
SERVER CERTIFICATE REJECTED: not signed by our trusted CA. Aborting.
client1@client1:~/phase_3$

client1@client1:~/phase_3$ hostname -I
10.0.2.6
client1@client1:~/phase_3$ ./server 5000
[12:10:22] [INFO ] Server listening on port 5000 ...

client1@client1:~/phase_3$ hostname -I
10.0.2.8
client1@client1:~/phase_3$ ./mitm_proxy 6000 10.0.2.6 5000 mallory_cert.pem
[MALLORY] Listening for victim on port 6000 ...
[MALLORY] Victim connected.
[MALLORY] Presenting my own certificate (not signed by the real CA)...
[MALLORY] Certificate sent. Waiting to see if victim sends a CHALLENGE (i.e. accepted my cert)...
[MALLORY] ATTACK FAILED: victim closed the connection without sending anything further.
[MALLORY] The victim's client rejected my certificate, exactly as intended by Phase 3.
client1@client1:~/phase_3$
```

Fig 11. Mallory presenting forged certificate , client rejects at certificate validation, with the exact error shown.

Stolen Certificate , No Private Key



```
client1@client1:~/phase_3$ hostname -I
10.0.2.5
client1@client1:~/phase_3$ ./client 10.0.2.8 6000 rajas
SERVER CERTIFICATE REJECTED: not signed by our trusted CA. Aborting.
client1@client1:~/phase_3$ ./client 10.0.2.8 6000 rajas
Server certificate validated. CN=ChatServer, signed by trusted CA. Proceeding.
^C
client1@client1:~/phase_3$ ./client 10.0.2.8 6000 rajas
Server certificate validated. CN=ChatServer, signed by trusted CA. Proceeding.
PROOF OF POSSESSION FAILED: server could not prove it holds the private key matching its certifi-
cate. Aborting.
client1@client1:~/phase_3$

client1@client1:~/phase_3$ hostname -I
10.0.2.8
client1@client1:~/phase_3$ ./mitm_proxy 6000 10.0.2.6 5000 certs/server_cert.pem mallory_
mallory_cert.pem mallory_key.pem
client1@client1:~/phase_3$ ./mitm_proxy 6000 10.0.2.6 5000 certs/server_cert.pem mallory_
mallory_cert.pem mallory_key.pem
client1@client1:~/phase_3$ ./mitm_proxy 6000 10.0.2.6 5000 certs/server_cert.pem mallory_key.pem
[MALLORY] Listening for victim on port 6000 ...
[MALLORY] Victim connected.
[MALLORY] Presenting a certificate (certs/server_cert.pem)...
[MALLORY] Certificate sent. Waiting to see if victim sends a CHALLENGE (i.e. accepted my cert)...
[MALLORY] My certificate was accepted! Victim sent a CHALLENGE: 4e5d2261cfab4f6b4df7e84c35f4f896
[MALLORY] I do NOT have the real server's private key, so I can only sign with my own.
[MALLORY] Forged proof sent. Waiting to see if victim proceeds to DH...
[MALLORY] ATTACK FAILED: victim closed the connection after the proof step.
[MALLORY] Proof-of-possession correctly caught that I don't hold the real private key.
client1@client1:~/phase_3$
```

Fig 12. Attacker presenting the real server certificate but signing with a different key - client rejects at proof of possession with exact error shown

Source Code

```
phase_3
├── cert.cpp
├── cert.h
├── certs
│   ├── ca_cert.pem
│   ├── ca_cert.srl
│   ├── ca_key.pem
│   ├── server_cert.pem
│   ├── server.csr
│   └── server_key.pem
├── client.cpp
├── crypto
│   ├── crypto.cpp
│   └── crypto.h
├── dh
└── dh.cpp
```

```
|
| |— dh_group_14.h
| |— dh.h
|— fake_server.cpp
|— fake_server_stolen_cert.cpp
|— Makefile
|— mitm_proxy.cpp
|— README.md
|— screenshots
| |— Legitimate Flow.png
| |— mallory certificate not accepted.png
| |— proof_of_possession_failed.png
|— server.cpp
|— setup_ca.sh
|— test_cert.cpp
|— test_pop.cpp
```

5 directories, 26 files

Phase 4 - End to End encryption Between clients

Design

Client to Client Key Exchange

C1 and C2 establish this second shared secret using the same Diffie-Hellman implementation already built for Phase 2, but run directly between the two clients rather than between a client and the server

This exchange is triggered by the `/e2e username` command: the initiating client generates a fresh DH keypair, unrelated to its existing client-server key, and sends its public value to the peer as the content of an ordinary chat message, tagged `__E2E_INIT__<hex public value>`.

The receiving client recognizes the `__E2E_INIT__` tag, generates its own DH keypair, computes the shared secret, and replies with its own public value tagged `__E2E_ACK__<hex>`.

Both clients then independently derive an AES-256 key from this shared secret, using the same hash-based key derivation function as Phase 2.

Message Wrapping

Once an end-to-end session has been established with a given peer, ordinary chat messages to that peer are protected by an additional, inner layer of encryption before being sent, rather than replacing the existing client-server encryption.

Sending

When a chat message is sent to a peer for whom an established E2E session exists, the message content is first encrypted using AES-256-GCM with the E2E-derived key.

This blob is hex-encoded and embedded as the content of an ordinary `MSG` frame, tagged `__E2E_MSG__<hex blob>`.

Receiving

On the receiving side, the client's existing outer-layer decryption (the Phase 2/3 client-server AES-GCM layer) proceeds as normal.

Server View

Verification

[illegible]

Fig 13. Matching fingerprint on both clients , computed independently.

```
client@client1:~/phase_4$ hostname -I
10.0.2.5
client@client1:~/phase_4$ ./client 10.0.2.6 5000 rajas
Server certificate validated. CN=ChatServer, signed by trusted CA. Proceeding.
Proof of possession verified - server genuinely holds the certificate's private key.
DH shared secret fingerprint: 1b1804566f7619b1
Registered as rajas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
>
[E2E] Session established with tejas. Fingerprint: e207ab2e3cc7c029
> /who
>
Online users: tejas
> /chat tejas
Now chatting with tejas
[tejas (E2E)] hello
>
[tejas (E2E)] recived your message
>

client@client1:~/phase_4$ hostname -I
10.0.2.7
client@client1:~/phase_4$ ./client 10.0.2.6 5000 tejas
Server certificate validated. CN=ChatServer, signed by trusted CA. Proceeding.
Proof of possession verified - server genuinely holds the certificate's private key.
DH shared secret fingerprint: 7ef4fc172bcdff04
Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
>
Online users: rajas
> /e2e rajas
E2E key exchange initiated with rajas
>
[E2E] Session established with rajas. Fingerprint: e207ab2e3cc7c029
> /chat rajas
Now chatting with rajas
hello
>
[rajas (E2E)] recived your message
>

[13:55:56] [DEBUG] Client: rajas, FD: 4
[13:56:02] [DEBUG] tejas requested /who -> [rajas]
[13:56:12] [INFO ] [RELAY] tejas -> rajas: _E2E_INIT_ 4A48E8659E68D39E0B4D3E9C802208E76D59AD4
6E38D944C9301DE42727769B7B682390898880C7842B1B7CE63ED71A5883EE416E18F4B35D1248B6DC4C04A60169B1
FBC4B3E968CE27C13FCE70B311DEB66B513B512F85D09DD91A87BD1056009ED42480EE7DA395FA8A05EFB0EEA2C9FF
2D54CF1869D03B841F7DF7DE6A438DA7949F9129B02A791705EEDFA2FF5F60C509AEA2ACE13ACF3B0E32FF703585E5
39B77E4E48859382F09DABAC1F577598112C4CF433E345E090DE17517F509BAF51F80ABCS859ED43423F4B72E3A6D0
DBF6403D133DE6F9A8A06689C76CA91C9DC8BA12D86C9858126F8A1522F273F433499381848B58342AE6A6C8A6392D
0B0
[13:56:12] [INFO ] [RELAY] rajas -> tejas: _E2E_ACK_ 9F35742F27FBB03884BFCC2978145927CC9798F
90E1203011F50890CD5583964FF8ED230E381EBE8A9783D0D62DA5998E6F99CE8C8D9EB7981F813F7444006542368
1B28B13A14423353812D1A15D3A21A6C83468F67D69CF8932D9824D19CC8001CE6E30025463B7973502D77CDDC099C
AEA6F6665DF00712A298203097765F0F67367B6703DAEBEBED7DE5D909F4FABFF3033DA9F3DCAEF5347316E500D44
9BA4AF79BDC00396C2E2FEB3209825431AEBFAS7D8CB3B794D8EC31FE41DEA668A8FE7448A4120E9B728038D3E9C
28B462789FE2A2874E0279F38A499E1FF8C6E1ABE94DC360F49B6E1E449EBA342D3A3555B169FC49B4DB8F2843F73
BC
[13:56:23] [DEBUG] rajas requested /who -> [tejas]
[13:58:26] [INFO ] [RELAY] tejas -> rajas: _E2E_MSG_ a87d74bc1f5ad1138e88fd2104cc6c9ee193f229
3d663ad6611071f08a71f573c5e0
[13:58:31] [INFO ] [RELAY] rajas -> tejas: _E2E_MSG_ f2488b396e5c32a426d9cf84fd057211dedaef28
b9f5203198d0e48d9ea2a8662c413c2f92797c79d558955113390959

```

Fig 14. Unreadable Messages by the server in the form of **__E2E__** , also showing correct decryption by the client

Source Code

```
phase_4
├── cert.cpp
├── cert.h
├── client.cpp
├── crypto
│   ├── crypto.cpp
│   └── crypto.h
├── dh
│   ├── dh.cpp
│   ├── dh_group_14.h
│   └── dh.h
├── Makefile
├── screenshots
│   ├── fingerprint_match_proof_1.png
│   ├── fingerprint_match_proof_2.png
│   └── message_E2E_log.png
├── server.cpp
└── setup_ca.sh
```

Phase 5 Forward Secrecy

Rekey Timer Design

Automatic Renegotiation

Once an initial end-to-end session is established via `/e2e`, a background thread is started per peer that sleeps for 60 seconds and then triggers a fresh key exchange, reusing the identical `__E2E_INIT__`/`__E2E_ACK__` handshake mechanism from Phase 4. This repeats for as long as the session remains active, producing a continuous sequence of independent keys rather than a single long-lived one.

Generation Numbers

Each key is tagged with a monotonically increasing generation number, starting at 1 for the initial handshake and incrementing by one with every subsequent rotation.

Grace period for the previous key

When a rotation completes, the outgoing key is replaced immediately for new messages but the *previous* generation's key is not discarded instantly.

It is retained for a short grace period (10 seconds) specifically to correctly decrypt any message that was encrypted and sent under the old key shortly before the rotation, but which happens to arrive at the receiver just after the rotation has already completed on that side.

Without this grace period, such a message would fail to decrypt purely due to timing.

After the grace period expires, the previous key is discarded and is no longer available for decryption under any circumstances.

Question on Collision Avoidance

Explain and justify your chosen approach for handling the case where both clients might attempt to trigger a renegotiation at approximately the same time.

For any given pair of communicating clients, the client whose username is lexicographically smaller is designated as the **sole initiator** of every key for that pair.

Verification

```

Sep 5 19:52
rajas@santhosh-MS-7C96:~$
Registered as rajas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
Online users: tejas
> /e2e tejas
E2E key exchange initiated with tejas
[14:20:29] [E2E] Key established with tejas (generation 1). Fingerprint: 86a5d4d93faea82
> /chat tejas
Now chatting with tejas
> hello
[tejas (E2E gen 1)] received message
[tejas (E2E gen 1)] waiting for new key generation
[14:22:09] [E2E] Key established with tejas (generation 2). Fingerprint: d274e48973839b21
> [tejas (E2E gen 2)] new key established testing message
> working message with new key
[ ]

Proof of possession verified - server genuinely holds the certificate's private key.
DH shared secret fingerprint: c77d48b1561b832d
Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> /who
[14:20:28] [E2E] Key established with rajas (generation 1). Fingerprint: 86a5d4d93faea82
> /who
Online users: rajas
> /chat rajas
Now chatting with rajas
[rajas (E2E gen 1)] hello
> received message
> waiting for new key generation
[14:22:07] [E2E] Key established with rajas (generation 2). Fingerprint: d274e48973839b21
> new key established testing message
[rajas (E2E gen 2)] working message with new keyt
[ ]

[0] 0:ssh*
```

Fig 15. A chat message sent immediately after a rotation , correctly decrypted

```

Sep 5 22:21
rajas@santhosh-MS-7C96:~$
Online users: tejas
> /e2e tejas
E2E key exchange initiated with tejas
[16:48:31] [E2E] Key established with tejas (generation 1). Fingerprint: fe5d1ba50481cb70
> /chat tejas
Now chatting with tejas
> hello can you receive me ?
[tejas (E2E gen 1)] yes receiving , waiting for automatic key generations
[16:49:34] [E2E] Key established with tejas (generation 2). Fingerprint: b3336b5299f52b87
> [tejas (E2E gen 2)] first key generated
> yes
[16:50:44] [E2E] Key established with tejas (generation 3). Fingerprint: 4387c12bcc198988
> 2nd key generated
[tejas (E2E gen 3)] okay bye
> /quit
client1@client1:~/phase_5$

Registered as tejas. Connected to 10.0.2.6:5000
Commands: @username message | /chat username | /who | /quit
> [16:48:30] [E2E] Key established with rajas (generation 1). Fingerprint: fe5d1ba50481cb70
> /chat rajas
Now chatting with rajas
> yes receiving , waiting for automatic key generations
[16:49:34] [E2E] Key established with rajas (generation 2). Fingerprint: b3336b5299f52b87
> first key generated
[rajas (E2E gen 2)] yes
[16:50:44] [E2E] Key established with rajas (generation 3). Fingerprint: 4387c12bcc198988
> [rajas (E2E gen 3)] 2nd key generated
> okay bye
> /quit
client1@client1:~/phase_5$

[0] 0:ssh*
```

Fig 16. At least two rotations shown, both clients with timestamps and fingerprints

Question on Forward Secrecy

What forward secrecy actually buys you that Phase 4 alone did not specifically in terms of what an attacker who later compromises a single key can and cannot do.

In Phase 4 alone, a single end-to-end key is established once via /e2e and then used for the entire remaining duration of the session, however long that happens to be. This creates a single point of catastrophic exposure: if an attacker ever obtains that one key and had also been recording the encrypted traffic all along, they could decrypt **every message from the entire conversation**, from the very first to the very last, all at once, using that one recovered key.

Phase 5's automatic key rotation changes what a compromise actually exposes. Because the session key changes to a genuinely new, independently derived value every 60 seconds, an attacker who compromises the key in use during, say, generation 7 gains the ability to decrypt only the messages that were sent during that specific ~60 second window.

Source Code

```
.
├── cert.cpp
├── cert.h
├── client
├── client.cpp
├── crypto
│   ├── crypto.cpp
│   └── crypto.h
├── dh
│   ├── dh.cpp
│   ├── dh_group_14.h
│   └── dh.h
├── Makefile
├── screenshots
│   ├── 2_key_generations.png
│   ├── rotated_key_creation_proof_1.png
│   └── rotated_key_creation_proof_2.png
├── server
├── server.cpp
├── server_key.pem
└── setup_ca.sh
```

4 directories, 22 files

Appendix A - Generative AI Usage

General / Setup

- Requested step-by-step guidance for setting up VirtualBox VMs, configuring NAT Network for inter-VM connectivity, port forwarding, SSH access between host and guest VMs, and using VirtualBox's Guest Additions / file sharing.
- Asked for help debugging VM networking issues (isolated NAT vs. NAT Network, IP conflicts, ping troubleshooting).
- Requested a Makefile to automatically compile all project binaries, and guidance reorganizing the project's file structure into `dh/`, `crypto/`, and `certs/` subfolders.

Conceptual / Background

- Asked for explanations of symmetric vs. asymmetric encryption, hybrid encryption, digital signatures, MACs, hash function security properties (preimage, second-preimage, collision resistance), and how they relate to each other.
- Requested a detailed walkthrough of Diffie-Hellman key exchange, including why a generator must be chosen carefully relative to the group's prime, and a worked toy example demonstrating a weak-generator vulnerability.
- Asked for the meaning and purpose of RFC 3526 Group 14, X.509 certificate fields (Subject, Issuer, Common Name, Organization), self-signed certificates, and the concept of a trust anchor.
- Requested explanations distinguishing certificate validation from proof-of-possession, and why neither alone is sufficient for authentication.
- Asked conceptual questions about C++ mechanics used in the implementation (e.g., `static_cast`, `MSG_NOSIGNAL`, signed/unsigned comparison pitfalls, thread safety, `std::mutex` usage).

Phase 1

- Asked for help identifying and fixing a bug where registered usernames retained a trailing newline character, breaking map lookups.
- Requested an explanation of the message-framing problem in TCP and implementation of length-prefixed framing as a fix, including a demonstration proving the original unframed version corrupted messages under rapid/bursty traffic.
- Asked for help diagnosing and fixing a bug where the server crashed entirely (via SIGPIPE) when a single client disconnected abruptly.
- Requested help capturing and interpreting Wireshark traffic across separate VMs.

Phase 2

- Requested a standalone tamper-detection test demonstrating AES-GCM correctly rejects a single-byte-modified ciphertext.
- Requested implementation of a MITM proxy performing two independent DH exchanges, and help interpreting the resulting evidence.

Phase 3

- Requested OpenSSL commands for setting up a Certificate Authority and issuing a server certificate, with verification via `openssl verify` and `s_client/s_server`.
- Asked for the MITM proxy to be updated to attempt the attack against the certificate-protected protocol, and debugging help when an initial version failed to correctly handle the new challenge/response step.

Phase 4

- Requested design guidance of a client-to-client Diffie-Hellman exchange tunneled through the existing encrypted client-server link, using the `__E2E_INIT__`/`__E2E_ACK__`/`__E2E_MSG__` wire tags.

Phase 5

- Asked for help diagnosing and fixing a bug where the rekey timer was only started on the client that manually issued `/e2e`, causing rotations to never occur when the other client was the designated initiator.

Appendix B - Git Commit History

Shell

```
rajas@santhosh-MS-7C96:~/Desktop/sem3/Network&Security/Assignment1$ git log --oneline
```

```
aa6ad7a (HEAD -> master, origin/master) screenshot proof added for rotation key
automatic generation , bug fixed timer
c74cbc2 60 seconds timer create
5527f53 screenshots added as proof for e2e fingerprint and message
c670297 setup e2e dh key generator client to client
dc017f5 added screenshots showing pop failed , and mallory certificate not
accepted
48559d8 added proof of possession after certificate validation logic
91a5180 Re organized similar files in their folders , created Makefile for easy
compilation
ac14f94 certificate creation and verification using openssl , integrating
certificate verification before dh setup in server and client cpp files
5bca533 mitm complete with screenshot and README
a15cfba mallory client face code complete , add global README and setup
screenshots
70d64ad encrypted registration and message with AES-GCM and screenshots added
87186eb dh handshake completed each client with server
e422642 Implemented dh key exchange
2f639f9 modified README for phase_1
6342eaa Added follow TCP Stream ss
2e788fb Phase 1 complete attached screenshots
54a74f8 solved message length prefix problem with mutex
62cdb04 client added and server modified to do proper logging and make the
endpoints while chatting like chat , who , quit
a9ecf3e server working with unlimited clients
```